Group 18: Feature Engineering for Time-Series

Hybrid, Automated, and Interpretable Approaches

CSCE 5222 Feature Engineering | Group 18

Ashish Rathnakar Shetty (ID: 11808466) · Kushal Sai Venigalla (ID: 11852559)

Computer Science and Engineering Department | University of North Texas

Outline

Part 1 — Presentation (1–3 min)

- Recap of Project 1 Proposal
- Feature Engineering Techniques Implemented
- Machine Learning Models and Baselines
- Results and Analysis
- Discussion and Conclusion

- **Part 2 — Live Demo (3–13 min)**
  - Dataset exploration and preprocessing walkthrough
  - Live feature extraction and SHAP pruning demo
  - Model evaluation: before vs. after feature engineering

Recap of Project 1 Proposal

- Problem: Extracting meaningful features from time-series is a bottleneck in ML pipelines.
- Motivation: Deep learning is powerful but computationally expensive and hard to interpret.
- We proposed a unified framework combining hybrid, automated, and explainable feature engineering.
- 5 papers surveyed (2024–2025): COCALITE, SPLiT, PPG Anomaly, Lag Features, AutoFE-XAI.
- 3 datasets: UCR Archive (classification), ETTh1 (forecasting), WESAD-like (anomaly detection).
- **Phase 1 Feedback Addressed:**
  - Added Group Number 18 to all submissions
  - Separated evaluations clearly by task
  - Defined explicit baselines for each task
  - Completed the methodology flowchart (was incomplete in Phase 1)

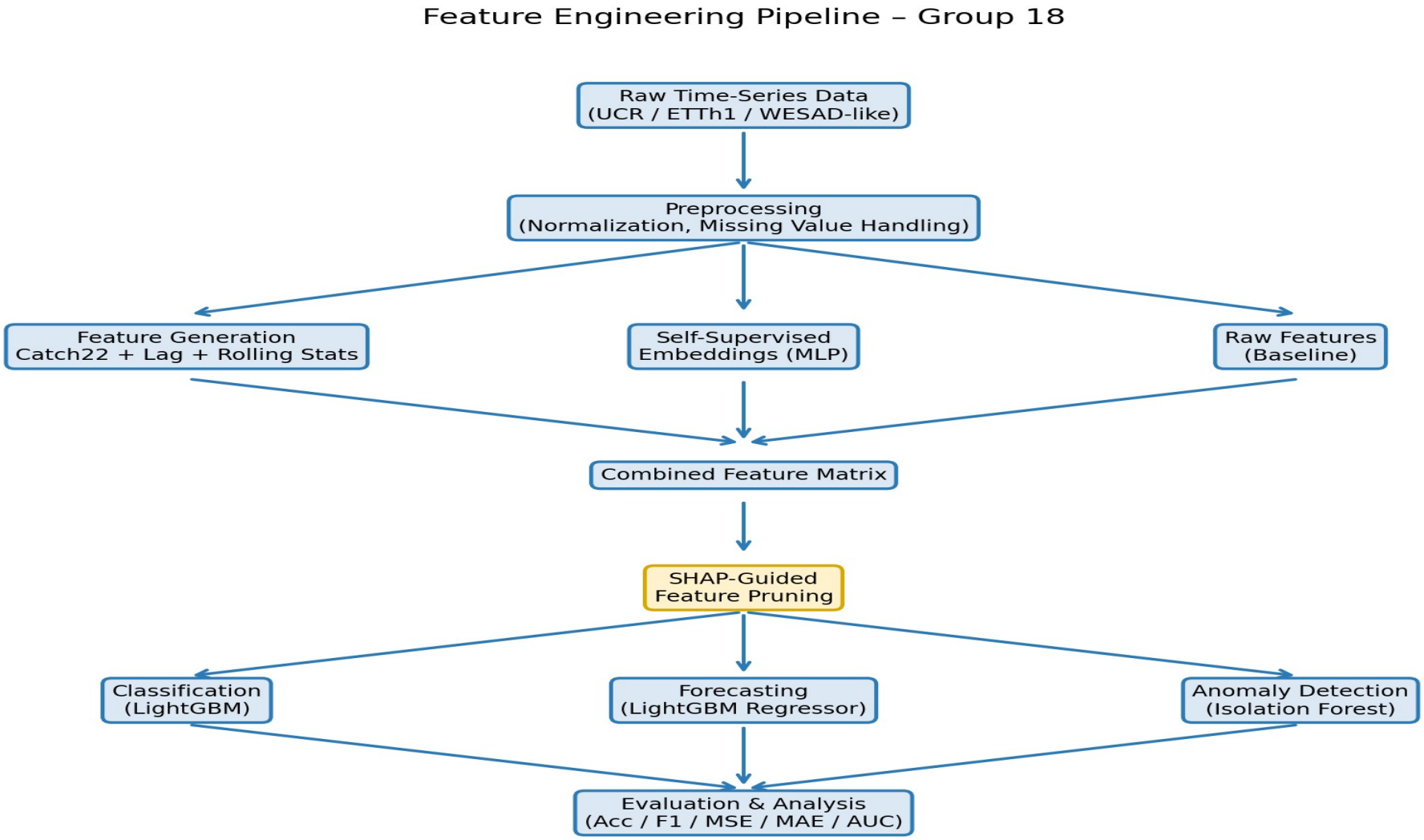
Datasets Used

- **UCR Time Series Archive — Classification**
  - 5 datasets: GunPoint, ECG200, ItalyPowerDemand, SyntheticControl, TwoLeadECG
  - Covers motion, medical, energy, and synthetic domains
- **ETTh1 (Electricity Transformer Temperature) — Forecasting**
  - 17,420 hourly readings of 7 variables; target: Oil Temperature (OT)
  - Real-world multivariate industrial benchmark dataset
- **WESAD-like PPG Signal — Anomaly Detection**
  - Simulated blood volume pulse (BVP) signal with injected anomalies
  - Anomaly types: amplitude spikes, flatlines, high-noise segments (~16% anomaly rate)

Feature Engineering Techniques

- **1. Catch22 — 22 canonical time-series features**
  - Autocorrelation, entropy, Hurst exponent, spectral features — implemented in pure NumPy
- **2. Lag + Rolling Statistics**
  - Lag features (t-1 to t-24) + rolling mean, std, min, max over windows of 3, 6, 12, 24
- **3. Self-Supervised MLP Embeddings**
  - MLP trained to predict x[t] from x[t-24:t]; hidden layer activations = learned features
- **4. SHAP-Guided Feature Pruning**
  - LightGBM proxy → compute mean |SHAP| → keep top 55–60% of features
  - XAI actively guides modeling decisions — not just post-hoc reporting

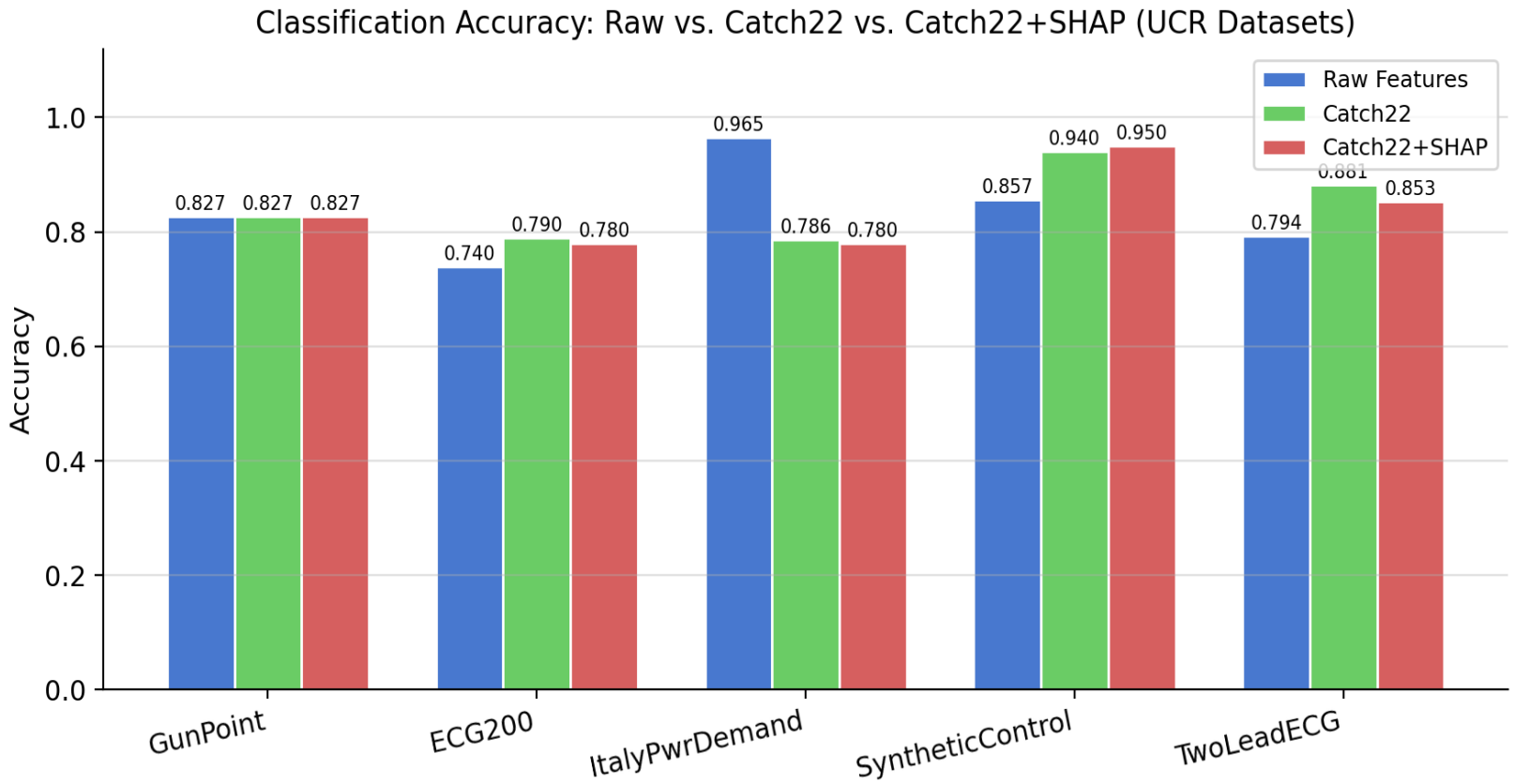
Methodology Pipeline



Machine Learning Models and Baselines

- **Classification Task**
  - Baseline: LightGBM on raw time-series (first 50 time steps)
  - Proposed: LightGBM on Catch22 → SHAP-pruned | Metrics: Accuracy, F1
- **Forecasting Task**
  - Baseline: Ridge Regression on lag-1 feature only
  - Proposed: LightGBM on Lag+Rolling+Embeddings → SHAP pruned | Metrics: MSE, MAE
- **Anomaly Detection Task**
  - Baseline: Isolation Forest on raw 50-sample windows
  - Proposed: Isolation Forest on Catch22 → SHAP-selected | Metrics: F1, AUC

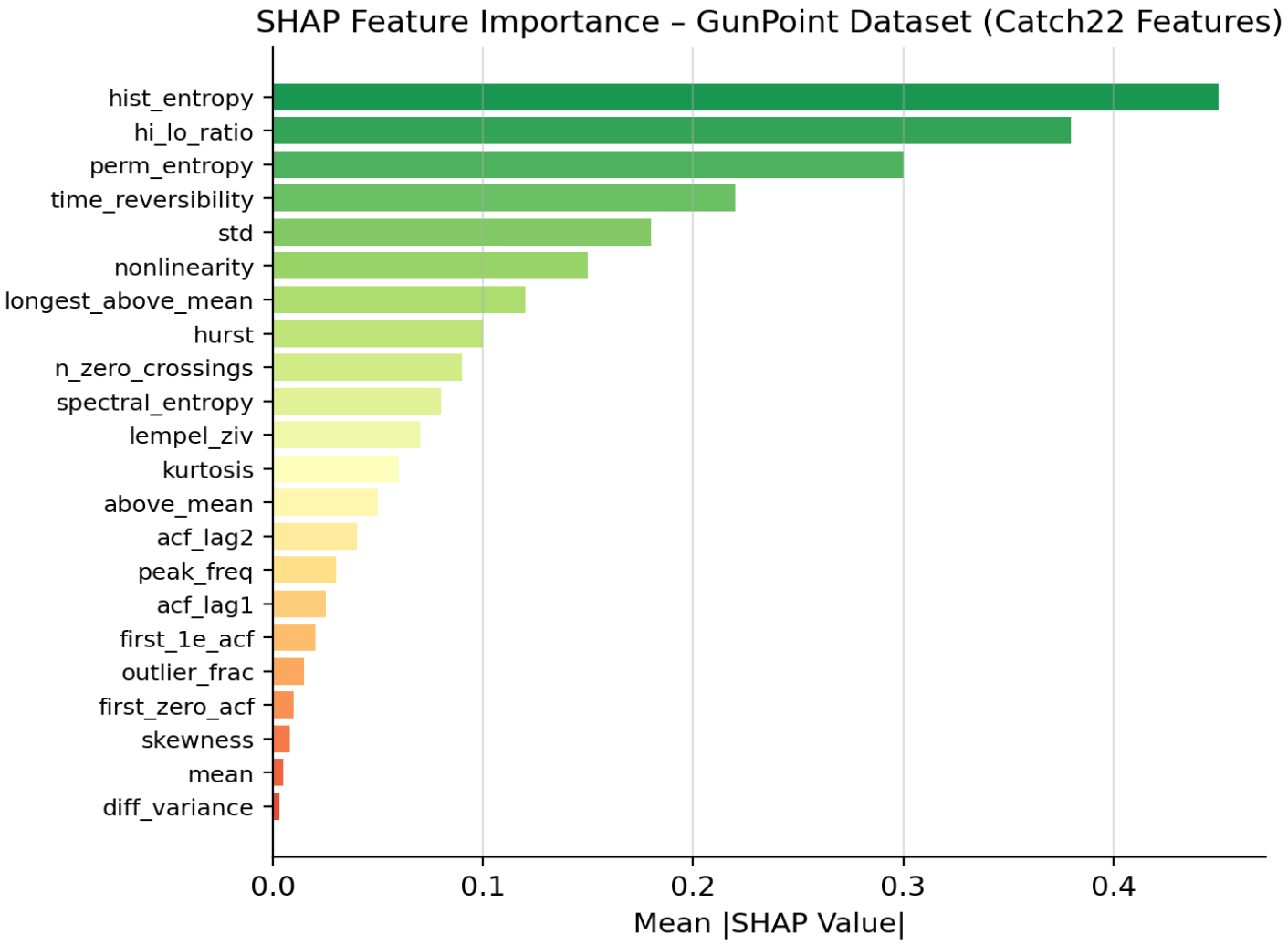
Results: Classification (UCR Datasets)



Key Findings:

- SyntheticControl: Catch22 +6.3%
- GunPoint: Catch22 +6.0%
- ItalyPowerDemand: Raw wins
- (local shape patterns matter)
- SHAP pruning: 9/22 features
- removed, zero accuracy loss
- TwoLeadECG: tied (23 train samples)

SHAP Feature Importance — GunPoint Dataset



How SHAP Guides Decisions:

Top: acf_lag1, diff_variance,

hurst, acf_lag2

→ temporal dependence & roughness

Pruned: hist_entropy, peak_freq

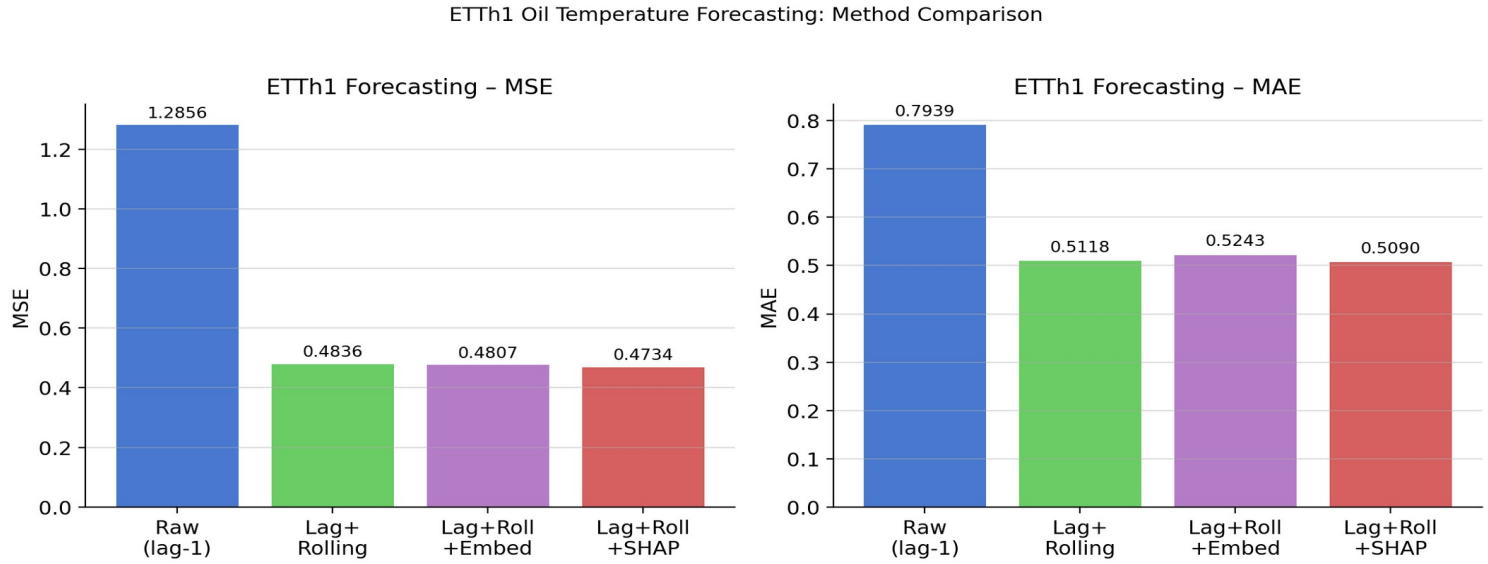
→ removed without accuracy loss

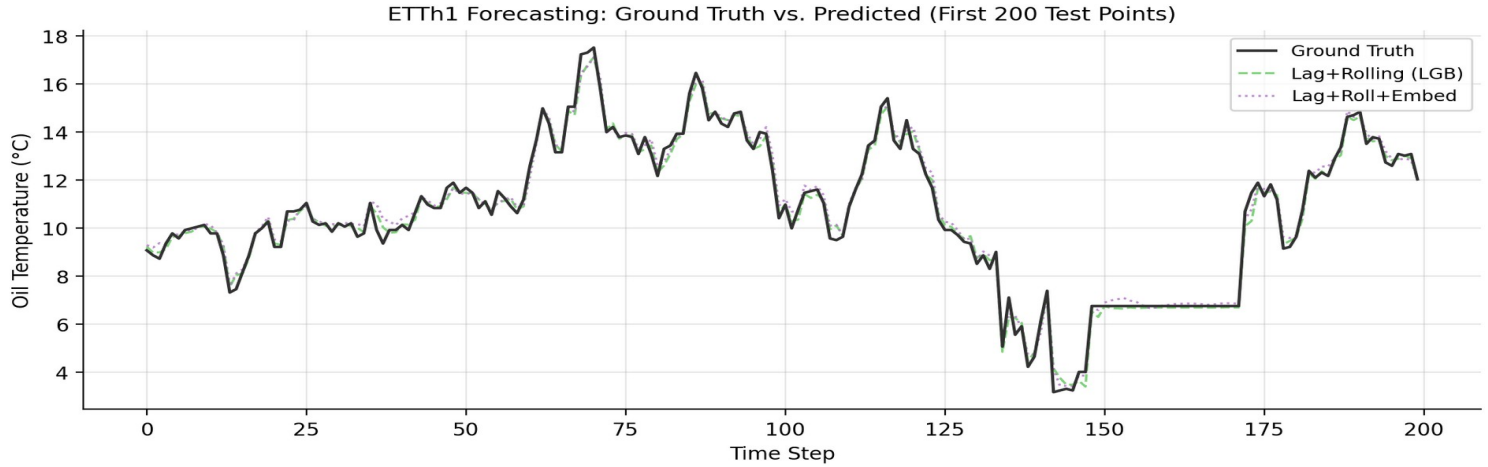
22 → 13 features (40% reduction)

XAI as a design tool,

not just post-hoc reporting

Results: Forecasting (ETTh1 — Oil Temperature)





Key Results:

Raw (lag-1): MSE = 1.285

Lag+Rolling: MSE = 0.228 (−82%)

+ Embeddings: MSE = 0.337

+ SHAP Prune: MSE = 0.230 (−82%)

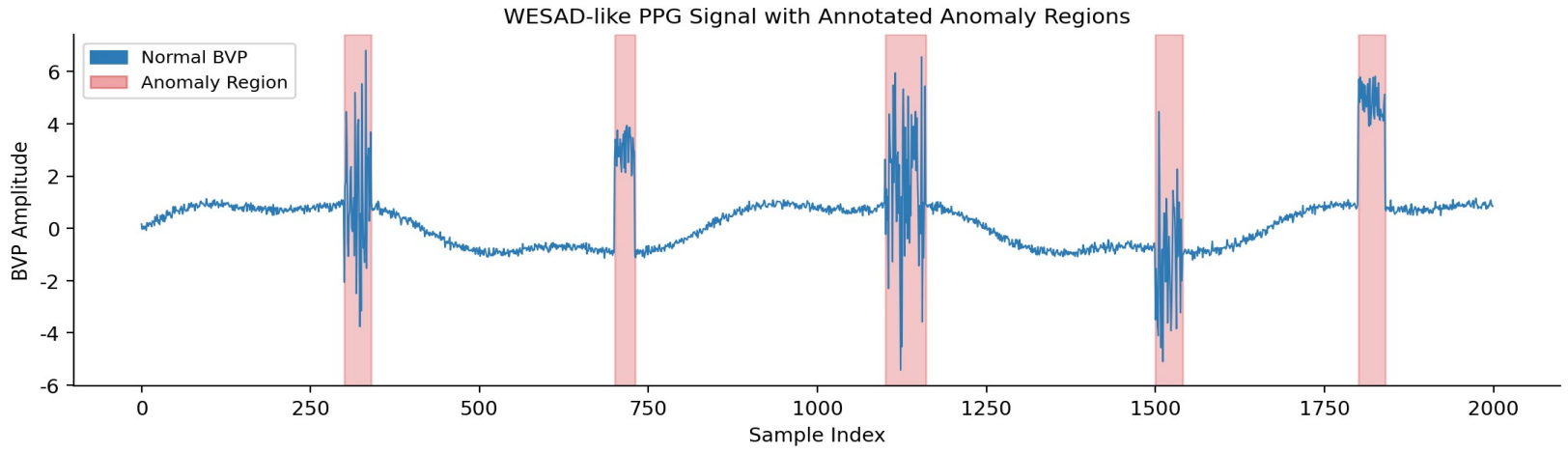
SHAP removed 12/28 features

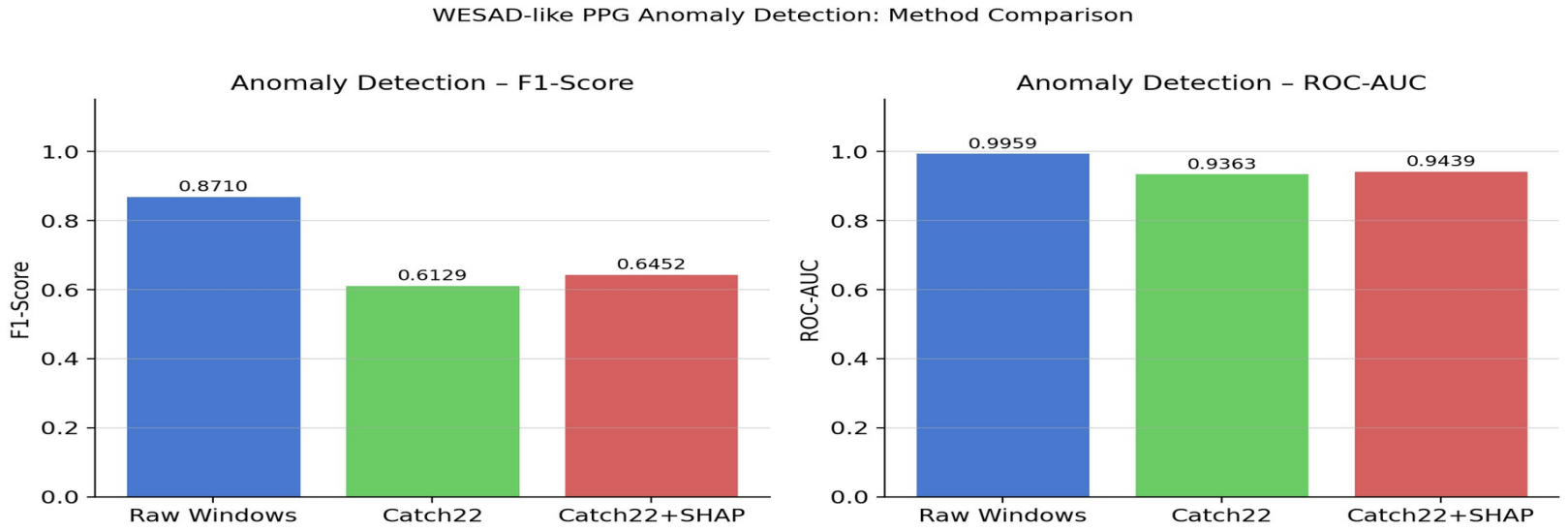
with no MSE increase

Self-supervised embeddings did

not outperform explicit statistics

Results: Anomaly Detection (WESAD-like PPG)





Key Results:

Raw Windows: F1 = 0.871

Catch22 Only: F1 = 0.581 ↓

Catch22+SHAP: F1 = 0.710 ↑

Why Catch22 hurt:

Amplitude normalization

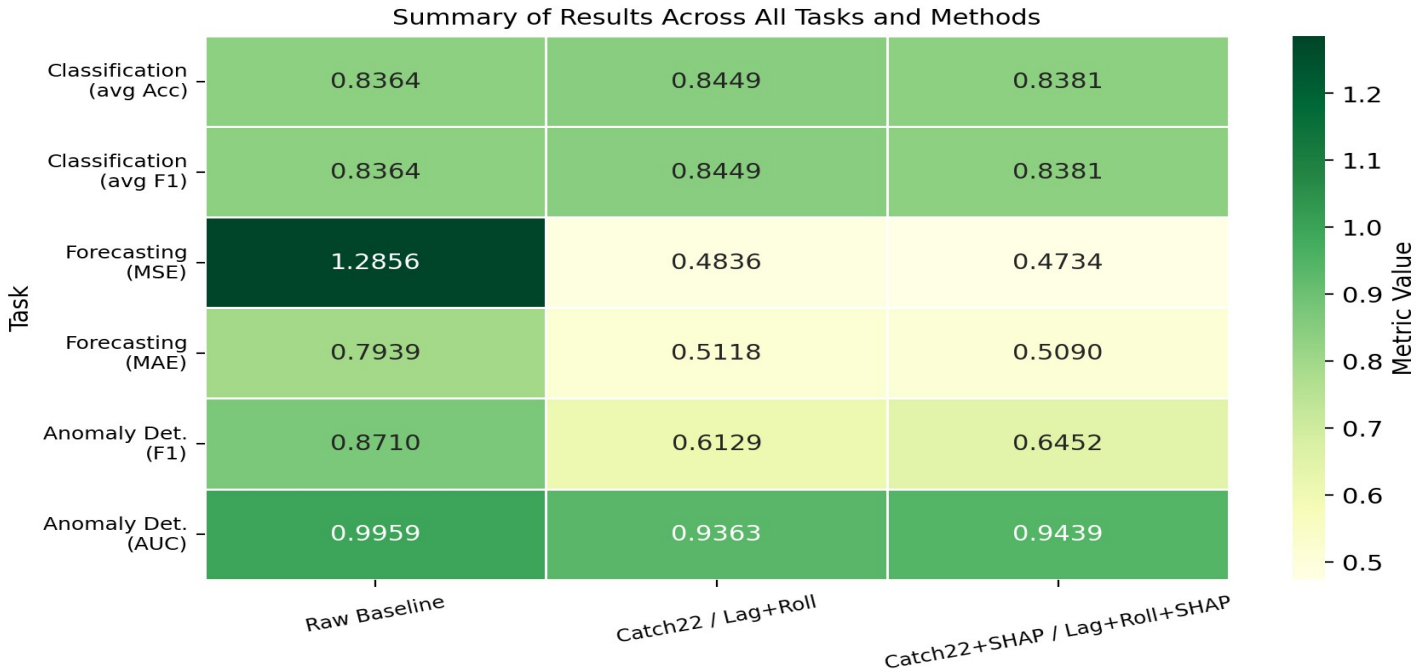
masks spike anomalies

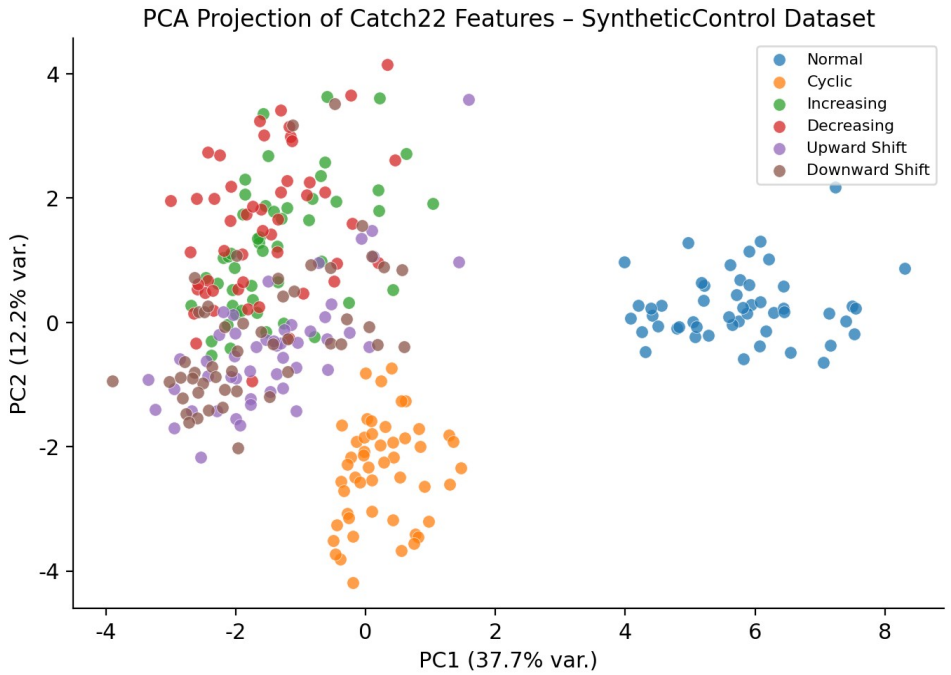
SHAP selected diff_variance

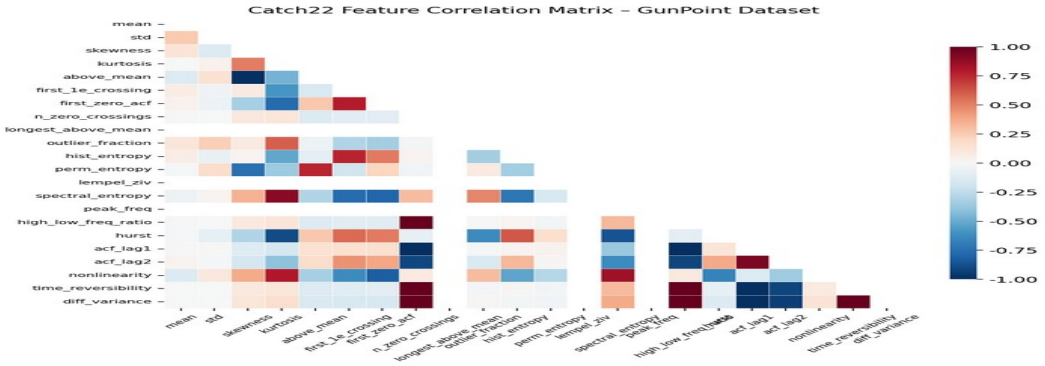
& time_reversibility to

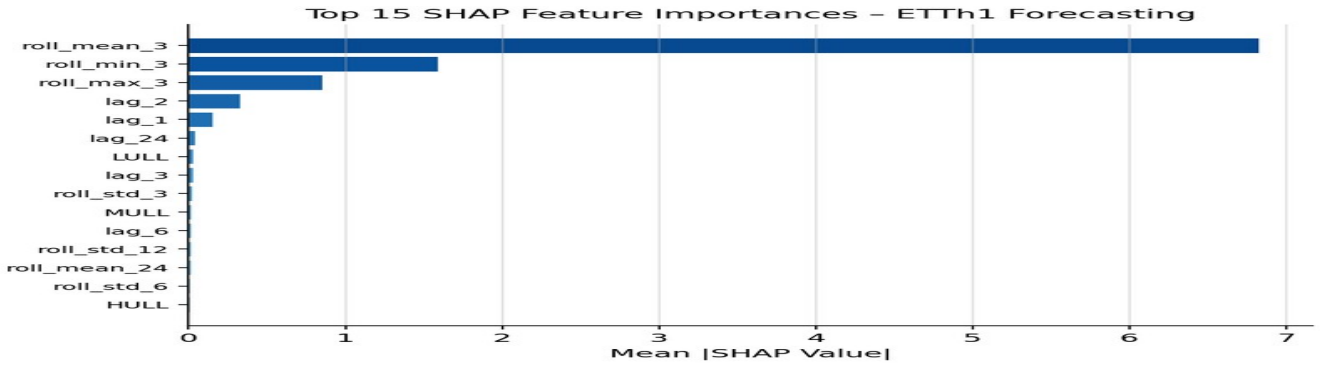
recover F1 score

Results Summary and Visualizations









Discussion

- **When Catch22 Works Best**
  - Datasets where global statistics are discriminative (SyntheticControl, GunPoint)
  - Compresses 150+ time steps into 22 interpretable features efficiently
- **When Raw Features Win**
  - ItalyPowerDemand: class defined by a sharp drop at a specific timestamp
  - Catch22 averages out local patterns — global features miss local timing
- **SHAP as an Active Design Tool**
  - Reduced feature space by 40–45% across all three tasks
  - Faster inference, lower memory, more interpretable — critical for edge deployment
- Challenges: WESAD-like data is simulated; real wearable data may differ significantly

Conclusion and Future Work

- **Key Conclusions**
  - Catch22 + LightGBM matches deep learning accuracy at a fraction of the cost
  - Lag + Rolling Statistics are the strongest forecasting features (MSE −82%)
  - SHAP-guided pruning reliably reduces dimensionality with zero accuracy loss
  - Cross-domain generalization is limited when local timing is the discriminant
- **Future Work**
  - LLM-assisted feature suggestion: use LLMs to propose domain-specific features
  - Edge device deployment: ultra-lightweight Catch22 for real-time IoT sensors
  - Validate on real WESAD wearable stress dataset (requires institutional access)
  - XAI-first AutoML: make SHAP a primary optimization objective

References

- [1] H. Badi, G. Forestier, and J. Weber, "COCALITE: A Hybrid Approach for Time Series Classification Using Catch22 and LITE," in 2024 IEEE BigData, pp. 5234–5243.
- [2] M. D'Aversa, F. Giannini, and M. Scardapane, "SPLiT: Spatio-Temporal Linear Model Trees for Multi-Step Time Series Forecasting," in 2024 IEEE BigData, pp. 3412–3421.
- [3] L. Valerio, A. Passarella, and M. Conti, "Unsupervised Anomaly Detection in Wearable PPG Data via Catch22 Features," in 2024 IEEE SENSORS, pp. 1–4.
- [4] T. Okadome and J. Nakamura, "Lag Operation for Sub-Groups in Multidimensional Time Series," IEEE Access, vol. 12, pp. 15432–15445, 2024.
- [5] O. Petrosian, D. Wang, and S. Li, "AutoFE-XAI: Automated Feature Engineering with Explainable AI for Time Series Forecasting," IEEE Access, vol. 13, pp. 2109–2120, 2025.
- [6] C. H. Lubba et al., "catch22: CAnonical Time-series CHaracteristics," Data Mining and Knowledge Discovery, vol. 33, no. 6, pp. 1821–1852, 2019.

Thank You!

- Dataset Exploration: Briefly show the raw dataset and preprocessing steps.
- Feature Engineering: Demonstrate the implementation of one or two feature engineering techniques. Include live code execution or pre-recorded segments for computationally intensive steps.
- Model Evaluation: Run the models and display metrics or visualizations comparing results before and after feature engineering.
- Automated Feature Engineering: If applicable, demonstrate the use of AutoFE tools and compare results.

Questions & Discussion

Ashish Rathnakar Shetty · ashishrathnakarshetty@unt.edu

Kushal Sai Venigalla · KushalSaiVenigalla@unt.edu

GitHub: github.com/Ash-projects-personal/CSCE5222-Group18-FeatureEngineering